2] How do control structures in PL/SQL help in writing complex queries?

Ans:

Control structures in PL/SQL provide a way to introduce logic, decision-making, and loops in your database programming, enabling you to write more complex and flexible queries and programs. By using control structures, you can extend the capabilities of SQL, handle business logic more effectively, and ensure that database operations are executed under the right conditions.

Here's how control structures help in writing complex queries:

1. Conditional Logic (IF-THEN, IF-THEN-ELSE)

Purpose: Control structures like IF-THEN, IF-THEN-ELSE, and IF-THEN-ELSIF-ELSE allow you to execute specific code blocks based on certain conditions.

How it helps: This is particularly useful when different actions need to be taken depending on certain data values. For instance, you can check if a sales figure exceeds a target, and based on that, perform different updates or insertions into tables.

Example:

```
DECLARE
  salary NUMBER := 50000;
  bonus NUMBER;
BEGIN
  IF salary > 60000 THEN
```

```
    bonus := salary * 0.10;
  ELSIF salary > 40000 THEN
    bonus := salary * 0.05;
  ELSE
    bonus := salary * 0.02;
  END IF;

  DBMS_OUTPUT.PUT_LINE('Bonus: ' || bonus);
END;
```

2. Looping (LOOP, WHILE LOOP, FOR LOOP)

Purpose: Looping control structures allow you to repeat a set of operations multiple times, either until a condition is met (e.g., WHILE) or for a specified number of iterations (e.g., FOR).

How it helps: Complex queries may require processing large datasets, updating multiple rows, or performing repeated calculations. Instead of executing multiple individual queries, you can loop through records and apply operations efficiently.

Example:

```
DECLARE
  CURSOR emp_cursor IS
    SELECT emp_id, salary FROM employees WHERE department = 'Sales';
  new_salary employees.salary%TYPE;
BEGIN
  FOR emp_record IN emp_cursor LOOP
    new_salary := emp_record.salary * 1.10;  -- Applying a 10% raise
```

```
    UPDATE employees
    SET salary = new_salary
    WHERE emp_id = emp_record.emp_id;
  END LOOP;

  COMMIT;
END;
```

3. Managing Transactions with Control Structures (COMMIT, ROLLBACK)

Purpose: Control structures allow you to group multiple DML operations (e.g., INSERT, UPDATE, DELETE) together as part of a transaction.

How it helps: This is important when you need to ensure that all changes are either committed or rolled back as a single unit. The COMMIT and ROLLBACK commands, often controlled by logic such as IF statements, ensure consistency in your database by confirming that a set of operations either all succeed or none are applied.

Example:

```
DECLARE
  from_balance NUMBER;
  to_balance NUMBER;
BEGIN
  -- Get the balances
  SELECT balance INTO from_balance FROM accounts WHERE account_id = 101;
  SELECT balance INTO to_balance FROM accounts WHERE account_id = 102;
```

```
  -- Ensure sufficient funds before transferring
  IF from_balance >= 500 THEN
    -- Deduct from one account and add to the other
    UPDATE accounts SET balance = balance - 500 WHERE account_id = 101;
    UPDATE accounts SET balance = balance + 500 WHERE account_id = 102;

    COMMIT;
  ELSE
    DBMS_OUTPUT.PUT_LINE('Insufficient funds');
    ROLLBACK;
  END IF;
END;
```

4. Error Handling (EXCEPTION Handling)

Purpose: Control structures in PL/SQL allow you to handle errors that may occur during query execution. You can use EXCEPTION blocks to catch and respond to exceptions (e.g., NO_DATA_FOUND, TOO_MANY_ROWS).

How it helps: When writing complex queries, errors may arise due to data issues (e.g., trying to update a non-existent record). The EXCEPTION block helps manage such errors gracefully without interrupting the program flow.

Example:

```
DECLARE
  total_sales NUMBER;
BEGIN
```

```
  SELECT SUM(sales_amount) INTO total_sales FROM sales WHERE month
= 'January';


  DBMS_OUTPUT.PUT_LINE('Total Sales for January: ' || total_sales);
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No sales data found for January');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An error occurred');
END;
```

5. Combining Control Structures to Write Complex Logic

Control structures allow you to combine different forms of logic in one PL/SQL block. You can use multiple IF-THEN conditions inside a LOOP, use nested FOR loops, or combine error handling with transactional control to implement more sophisticated logic.

Example:

Imagine you want to process payments for all orders, but only for customers who have not exceeded their credit limit. You can combine multiple control structures to write such logic.

Example:

```
DECLARE
  CURSOR order_cursor IS
   SELECT order_id, customer_id, total_amount FROM orders WHERE
status = 'Pending';
```

```
  credit_limit NUMBER;
  current_balance NUMBER;
BEGIN
  FOR order_record IN order_cursor LOOP
    -- Check the customer's credit limit
    SELECT credit_limit, current_balance INTO credit_limit, current_balance
    FROM customers WHERE customer_id = order_record.customer_id;

    IF current_balance + order_record.total_amount <= credit_limit THEN
      -- Proceed with payment
      UPDATE orders SET status = 'Paid' WHERE order_id =
order_record.order_id;
      COMMIT;
    ELSE
      -- If credit limit exceeded, skip this order
      DBMS_OUTPUT.PUT_LINE('Credit limit exceeded for order: ' ||
order_record.order_id);
      ROLLBACK;
    END IF;
  END LOOP;
END;
```